

这是已分享的 ChatGPT 对话副本
举报对话

我有个想法，就是做一个新型的个人博客，主要是我观察到了目前很多人都在使用llm生成的html作为笔记，那么为什么不能开发一种新型的个人博客项目更加便捷的帮助用户host html笔记，并分享给其他人呢？这个项目还应该接入一种llm接口方便llm读取网站内容，这个项目应该作为开源项目，帮我扩展一下想法

我会把这个想法扩展成一个可落地的开源项目设想：先定位现有生态空白，再拆成产品形态、核心功能、技术架构、开源策略和早期路线图。

已思考 41s >

你的观察很有价值。这个项目不应只是“又一个博客系统”，而应定位为：

面向 AI 时代的 HTML 笔记发布与读取基础设施。

人类可以像看博客一样浏览，LLM/Agent 可以像读结构化知识库一样读取、引用、搜索和重组内容。

1. 核心判断：机会点在哪里

现有工具大致分三类：

第一类是传统静态站点或数字花园，例如 Quartz 可以把 Markdown 内容生成网站，定位于发布 digital garden；Docusaurus 也适合文档、博客和项目网站。 Quartz +1

第二类是笔记产品或自托管时间线，例如 Obsidian 面向本地 Markdown 知识库，Memos 面向轻量、自托管、Markdown-native 的快速记录。 Obsidian +1

第三类是部署/搜索基础设施，例如 Cloudflare Pages、Pagefind，分别解决静态站部署和静态站全文搜索问题。 Cloudflare Pages +1

但你说的场景有一个新变量：**LLM 生成的 HTML 笔记越来越多**。这类内容不一定是 Markdown，不一定适合传统博客编辑器，往往包含复杂排版、交互组件、可视化、图表、CSS、甚至小型单页应用。现有博客系统并不是为“保存、托管、版本化、分享和让 AI 读取这种 HTML artifact”设计的。

所以这个项目的切入点可以是：

一个专门托管 LLM-generated HTML notes 的开源个人发布系统。

它的目标用户不是“想写博客的人”，而是：

- 经常用 ChatGPT、Claude、Gemini、Cursor 等工具生成 HTML 笔记的人；
- 希望把 AI 生成的研究报告、学习卡片、交互页面、可视化笔记保存下来的人；
- 想把个人知识库暴露给其他 LLM/Agent 读取的人；
- 想要比 Notion/博客/静态站更轻、更开放、更 AI-native 的发布方案的人。

2. 项目的一句话定位

可以这样定义：

OpenNote HTML：一个开源的 AI-native 个人博客/知识发布系统，让用户一键托管 HTML 笔记，并自动生成适合人类和 LLM 阅读的双通道内容。

更产品化一点：

GitHub Pages for AI-generated HTML notes, with built-in LLM-readable APIs.

中文表述：

给 AI 生成的 HTML 笔记用的开源博客系统。

这个定位很清晰，也容易传播。

3. 关键差异化：不是博客，而是“HTML artifact garden”

传统博客的核心对象是“文章”。

你的项目的核心对象应该是“HTML artifact”。

一个 artifact 可以是：

- 一篇普通文章；
- 一个交互式学习笔记；
- 一个数据可视化页面；
- 一个 AI 生成的研究简报；
- 一个课程总结页；
- 一个项目说明书；
- 一个带搜索、折叠、图表、代码块的单页知识卡片；
- 一个完整 mini app。

所以内容模型不要只设计成 Post，而应设计成：

```
Artifact
- title
- slug
- description
```



```
- html_file
- extracted_text
- generated_markdown
- metadata
- tags
- visibility
- license
- source_prompt optional
- source_model optional
- created_at
- updated_at
- version_history
- ai_reading_policy
```

这样项目天然区别于 Hexo、Hugo、WordPress、Quartz、Docusaurus。

4. 人类阅读层 + LLM 阅读层

这是整个项目最重要的设计。

A. 人类阅读层

给普通访客看的内容：

- 个人主页；
- 笔记列表；
- 标签/专题；
- 搜索；
- RSS；
- 深色模式；
- 代码高亮；
- 评论或 Webmention；
- 每篇 HTML 笔记独立链接；
- 可嵌入 iframe；
- 支持自定义域名；
- 支持公开、私密、密码、unlisted。

B. LLM 阅读层

给 AI/Agent 读取的内容：

- /llms.txt
- /sitemap.xml
- /feed.json
- /api/artifacts
- /api/artifacts/:slug
- /api/search?q=...

- `/api/chunks?artifact=...`
- `/mcp` 或 MCP server

`/llms.txt` 是一个已经被提出的标准化思路，用于帮助 LLM 在推理时理解网站内容；它本质上更像“给 LLM 的网站内容地图”，不是强制爬虫控制协议。 [llms-txt +1](#)

MCP 则可以作为更进一步的 Agent 接口。官方文档将 MCP 描述为一种开放标准，用于让 AI 应用连接外部系统、数据源和工具。 [Model Context Pr... +1](#)

也就是说，你可以分两层做：

基础版：

`/llms.txt`
`/sitemap.xml`
`/api/artifacts`
`/api/search`

高级版：

MCP server
RAG chunks API
semantic search API
agent write/publish API



5. 最有价值的功能模块

5.1 一键发布 HTML 笔记

用户可以通过几种方式发布：

- 上传 `.html` 文件；
- 粘贴 HTML；
- 从 Git repo 同步；
- CLI 发布：`htmlnote publish report.html`
- 浏览器插件发布当前页面；
- API 发布；
- MCP tool：让 LLM 直接调用 `publish_html_note`。

这会形成一个很强的闭环：

用户让 LLM 生成 HTML 笔记

- 一键发布到个人站
- 自动生成 AI 可读版本
- 其他 LLM 可以读取、搜索、引用



5.2 HTML 安全沙箱

这是项目成败的关键之一。用户上传 HTML，里面可能有 JS、iframe、外链脚本、恶意代码。

建议设计三种渲染模式：

Safe Mode

- 默认模式
- 移除危险脚本
- 只允许安全 HTML/CSS
- 适合普通文章和笔记

Sandbox Mode

- 原样保留 HTML/JS
- 通过 `iframe sandbox` 隔离
- 限制 `top navigation`、`cookies`、`storage`、`external requests`

Trusted Mode

- 用户自己部署时可以开启
- 允许更完整的交互能力

还需要：

- CSP；
- `iframe sandbox`；
- HTML sanitization；
- 资源代理；
- 限制外链脚本；
- 限制上传大小；
- 防止 phishing；
- 防止把站点变成垃圾内容农场。

5.3 自动抽取 AI-readable 内容

每篇 HTML 上传后，系统自动生成：

<code>original.html</code>	原始 HTML
<code>rendered.html</code>	公开展示版本
<code>content.md</code>	抽取出的 Markdown
<code>content.txt</code>	纯文本
<code>metadata.json</code>	标题、摘要、标签、时间、作者、许可证
<code>chunks.json</code>	供 RAG 使用的分块
<code>embeddings optional</code>	向量索引

这样 LLM 不需要解析复杂 HTML，而是直接读取干净内容。

示例：

```
{
  "title": "Transformer Architecture Notes",
  "url": "https://example.com/notes/transformer",
  "summary": "A visual explanation of attention, positional encoding, and encoder-decoder",
  "tags": ["ai", "transformer", "deep-learning"],
  "formats": {
    "html": "/notes/transformer",
    "markdown": "/notes/transformer.md",
    "text": "/notes/transformer.txt",
    "json": "/api/artifacts/transformer"
  }
}
```

5.4 LLM 读取策略

用户应该能控制 AI 是否可以读取自己的内容。

每篇笔记可以设置：

```
AI readable: yes / no
AI indexable: yes / no
Allow summarization: yes / no
Allow training use: no preference / disallow notice
Preferred citation: title + url + author
License: CC BY / CC BY-NC / All rights reserved / MIT for code
```



这比普通博客更符合未来内容发布需求。

5.5 双重搜索：人类搜索 + Agent 搜索

人类搜索可以用 Pagefind 这类静态搜索方案；Pagefind 可以在静态站构建后生成搜索 bundle，并提供 JS search API。 [Pagefind](#)

Agent 搜索则可以返回结构化结果：

```
[
  {
    "title": "Attention Mechanism Explained",
    "url": "...",
    "snippet": "...",
    "score": 0.83,
    "format": "markdown",
    "updated_at": "2026-05-20"
  }
]
```

未来可以再加向量搜索，但 MVP 不一定需要。

6. 可能的技术架构

方案 A：纯静态优先

适合个人用户、自托管、GitHub Pages、Cloudflare Pages。

```
CLI / Web UI
  ↓
HTML ingest
  ↓
sanitize + extract text + generate metadata
  ↓
static build
  ↓
deploy to Pages / Vercel / Netlify / GitHub Pages
```



优点：

- 成本低；
- 易部署；
- 开源项目容易被接受；
- 用户数据可完全自有；
- 不依赖中心化服务。

缺点：

- 动态权限、私密笔记、评论、协作较难。

方案 B：轻量服务端

适合 SaaS 或自托管。

```
Frontend: Next.js / SvelteKit / Astro
Backend: Node.js / Go / Rust
Storage: S3/R2/MinIO
DB: Postgres / SQLite / D1
Search: Pagefind / Meilisearch / Tantivy
Queue: background worker
LLM interface: REST + MCP
```



优点：

- 上传体验好；
- 支持账号、权限、私密内容；
- 支持 API；
- 支持多用户。

缺点：

- 运维复杂；
- 开源自部署门槛更高。

我建议的路线

先做 **静态优先 + 可选服务端**。

核心包：

@project/core	HTML 解析、清洗、metadata、导出
@project/cli	本地发布
@project/theme	默认博客主题
@project/server	可选服务端
@project/mcp	MCP server
@project/web	管理后台



这样既适合开源社区，又方便未来商业化。

7. MVP 应该怎么做

第一版不要做太大。建议 MVP 只解决一个强需求：

我有一个 HTML 文件，我想 30 秒内把它变成一个可访问、可分享、可被 LLM 读取的个人笔记页面。

MVP 功能：

1. 上传或导入 HTML；
2. 自动生成标题、摘要、标签；
3. 安全预览；
4. 发布为公开链接；
5. 自动生成 .md 、 .txt 、 .json ；
6. 自动维护 /llms.txt ；
7. 支持全站搜索；
8. 支持自定义主题；
9. 支持 GitHub/Cloudflare Pages 部署；
10. 支持导出完整静态站。

先不要做：

- 多人协作；
- 评论系统；
- 复杂权限；
- 复杂编辑器；
- 内置大模型生成；
- 社交网络；
- 移动端 App。

这些会拖慢项目。

8. 项目的概念命名

可以考虑几个方向：

偏技术感

- HTMLGarden
- ArtifactPress
- NoteHost
- AIGarden
- PageVault
- LLMPress
- OpenArtifact
- PromptPages

偏个人知识库

- MindPages
- GardenHTML
- NoteGarden
- Personal Pages
- Knowledge Frames

偏 AI-native

- AgentReadable
- LLM Garden
- AI Pages
- ContextPress
- ContextGarden

我比较推荐：

ArtifactPress

原因是它避开“blog”的旧心智，强调“artifact”。AI 生成的 HTML 页面本来就更像 artifact，而不是传统 post。

副标题：

Open-source publishing for AI-generated HTML notes.

9. 开源项目结构

可以这样设计 repo:

```
artifactpress/  
  apps/  
    web/                管理后台  
    demo-site/          示例站点  
  packages/  
    core/               HTML ingest、解析、清洗、导出  
    cli/                命令行工具  
    renderer/           iframe/sandbox 渲染  
    theme-default/      默认主题  
    llm/                llms.txt、markdown、chunk 生成  
    mcp-server/         MCP 接口  
    search/             搜索索引  
  templates/  
    minimal/  
    digital-garden/  
    research-notes/  
  docs/  
  examples/  
    ai-generated-report/  
    interactive-note/  
    visual-cheatsheet/
```



CLI 示例:

```
</> Bash  
  
artifactpress init my-notes  
cd my-notes  
artifactpress add ./transformer-note.html  
artifactpress build  
artifactpress dev  
artifactpress deploy
```



API 示例:

```
</> Bash  
  
curl -X POST https://your.site/api/artifacts \  
  -H "Authorization: Bearer xxx" \  
  -F "file=@note.html"
```



MCP tool 示例:

```
publish_html_note(title, html, tags, visibility)  
search_notes(query)  
read_note(slug, format="markdown")  
list_recent_notes(limit)
```



10. 最吸引人的产品体验

你可以把整个流程设计成这样：

1. 用户在 ChatGPT 里生成一个 HTML 学习笔记
2. 点击复制 HTML
3. 打开 ArtifactPress
4. 粘贴
5. 系统自动预览、抽取标题、摘要、标签
6. 用户点击 Publish
7. 得到一个公开链接
8. 系统同时生成：
 - 人类可看的页面
 - LLM 可读的 markdown/json
 - llms.txt 入口
 - 搜索索引



进一步，可以做一个浏览器书签脚本：

Save to ArtifactPress



或者做一个 VS Code/Cursor 插件：

右键 HTML 文件 → Publish as Note



这会非常贴合 LLM-heavy 用户。

11. 可以强化的“AI-native”特性

11.1 Prompt provenance

保留来源信息：

Generated by: ChatGPT / Claude / Gemini / local model
Prompt: optional
Model: optional
Created at: ...
Edited by human: yes/no



这对未来内容可信度很重要。

11.2 AI summary card

每篇笔记自动生成：

- TL;DR
- Key claims
- Concepts



- Prerequisites
- Related notes
- Suggested follow-up questions

11.3 引用友好

每篇笔记的段落、标题、代码块都可以生成稳定锚点：

```
/notes/transformer#attention-score
```



LLM 返回答案时可以引用这些稳定链接。

11.4 知识图谱

从 HTML/Markdown 抽取：

```
entities
concepts
links
tags
references
```



生成个人知识图谱。

11.5 Agent workspace

让 Agent 可以这样使用网站：

```
search my notes about "vector database"
read the most relevant note
summarize it
find related notes
generate a learning path
```



这就是普通博客做不到的。

12. 风险与解决方案

风险一：HTML 安全问题

这是最大风险。必须默认 sandbox，不要默认信任用户 HTML。

建议：

- 默认 iframe sandbox；
- 禁止任意外链脚本；
- 对公开页面做 CSP；
- 上传内容和主站域名隔离，例如：

- user.com 主站；
- artifacts.user.com 渲染 HTML；
- 对 SaaS 版本做滥用检测。

风险二：项目被理解成“静态博客生成器”

所以对外传播不要说“another blog engine”。

应该说：

A publishing layer for AI-generated HTML artifacts.



中文：

AI 生成 HTML 笔记的发布层。



风险三：LLM 接口标准变化快

llms.txt 目前仍是提案，不能把项目完全押注在它上面。更稳妥的策略是同时支持：

```
/llms.txt  
/sitemap.xml  
/feed.json  
REST API  
MCP server  
Markdown export
```



这样即使某个标准没有成为主流，项目仍然有价值。

风险四：用户为什么不用 GitHub Pages?

因为 GitHub Pages 解决的是“托管静态文件”，不是：

- 自动清洗 HTML；
- 自动生成 AI-readable 版本；
- 自动维护笔记索引；
- 自动生成 llms.txt；
- 自动搜索；
- 自动版本化；
- 一键从 LLM 发布；
- 给 Agent 读写接口。

你的项目应该站在 GitHub Pages/Vercel/Cloudflare Pages 之上，而不是和它们竞争。

13. 商业化可能性

虽然项目开源，但可以保留服务化空间。

开源免费

- CLI;
- 静态站生成;
- 基础主题;
- 本地构建;
- llms.txt;
- markdown/json export;
- self-host server。

托管版收费

- 一键部署;
- 自定义域名;
- 私密笔记;
- 大文件存储;
- 向量搜索;
- MCP endpoint;
- 团队空间;
- 访问分析;
- 自动备份;
- CDN;
- 高级主题。

面向开发者的付费点

- API quota;
- private MCP server;
- semantic search;
- team knowledge base;
- custom branding;
- organization workspace。

14. 早期路线图

Phase 0: 概念验证

目标：证明“HTML artifact → AI-readable note site”成立。

交付：

- CLI;
- 上传/导入 HTML;
- 生成静态站;
- 生成 Markdown/Text/JSON;

- 生成 `/llms.txt`；
- 一个默认主题。

Phase 1: 可用产品

目标：普通用户能持续使用。

交付：

- Web UI；
- 标签、搜索、主页；
- 安全沙箱；
- RSS；
- 自定义域名文档；
- GitHub Actions/Cloudflare Pages 部署模板。

Phase 2: AI-native

目标：让 Agent 真正能读。

交付：

- MCP server；
- semantic chunks；
- search API；
- note reading API；
- related notes；
- citation anchors。

Phase 3: 生态扩展

目标：成为 LLM 笔记发布标准层。

交付：

- 浏览器插件；
- VS Code/Cursor 插件；
- Obsidian 插件；
- Raycast 插件；
- ChatGPT/Claude 使用说明；
- 示例站点集合；
- 主题市场。

15. 我会建议你把项目收束成这个版本

不要一开始做“新型个人博客”。这个说法太宽，容易落入已有竞争。

更好的表述是：

一个开源的 AI-generated HTML note hosting system。

核心价值：

LLM 负责生成内容。
用户负责选择和整理。
ArtifactPress 负责托管、版本化、展示和让其他 LLM 读取。



最小产品主张：

把任何 HTML 笔记变成一个可分享、可搜索、可被 LLM 读取的个人知识页面。

这比“博客”更准确，也更有时代感。



来源

我觉得用户不一定只能上传html文件，也可以上传md或docx之类的笔记文件，然后让该项目调用llm转换成html，具有几种转换预设，用户也可以自定义转换

这个补充会显著扩大项目边界：它不再只是托管 HTML，而是“把各种个人笔记转换成可发布的 AI-native 网页”。重点应放在转换流水线、预设系统和可复现性上。

已思考 21s >

你的补充会让项目定位更强：它不应该只是 **HTML 笔记托管器**，而应该升级成：

AI-native 文档发布引擎：用户上传 Markdown、DOCX、HTML 等笔记文件，系统通过规则转换 + LLM 增强，把它们变成可发布、可搜索、可被 LLM 读取的 HTML artifact。

这样项目的价值链会完整很多。

1. 新定位：从 HTML Host 变成 Note-to-HTML Publishing Engine

原始定位是：

LLM 生成 HTML → 用户上传 → 托管和分享



现在可以扩展成：

用户已有笔记 / LLM 生成笔记 / 文档资料



↓
上传 md / docx / html / txt / pdf

↓
选择转换预设

↓
LLM + 模板系统转换成高质量 HTML 页面

↓
发布为个人博客 / 知识库 / artifact 页面

↓
同时生成 LLM-readable API

这会让项目覆盖更多真实场景。

—

- Markdown 笔记；
- Obsidian vault；
- Word 文档；
- 课堂笔记；
- 研究摘录；
- 论文阅读笔记；
- ChatGPT 输出的 Markdown；
- DOCX 报告；
- Notion 导出的 Markdown/HTML；
- 纯文本草稿；
- 未来也可以支持 PDF。

所以项目的核心对象可以从 HTML Artifact 扩展成：

Source Note → Rendered Artifact → LLM-readable Context



也就是：

- **Source Note**：用户原始文件，例如 .md 、 .docx 、 .html 。
- **Rendered Artifact**：转换后的可视化 HTML 页面。
- **LLM Context**：为 AI/Agent 准备的 Markdown、JSON、chunks、metadata、search index。

2. 核心产品能力：转换预设

转换预设会成为这个项目最有差异化的地方。

传统博客系统的流程是：

写文章 → 选择主题 → 发布



你的项目可以是：

上传笔记 → 选择转换风格 → 预览 → 局部修改 → 发布



预设可以分成几类。

2.1 干净文章模式

适合普通博客文章、随笔、技术文章。

输出特征：

- 标准文章布局
- 清晰标题层级
- 代码高亮
- 目录
- 引用块
- 脚注
- 阅读进度
- 移动端适配



适合输入：

.md
.docx
.txt
.html



2.2 学习笔记模式

适合课程笔记、读书笔记、概念整理。

输出特征：

- 重点概念卡片
- 定义块
- 例子块
- 易错点
- 总结区
- 复习问题
- 标签和知识点索引



这个模式特别适合 AI 生成的学习内容。

2.3 研究简报模式

适合论文阅读、市场研究、技术调研。

输出特征：

- TL;DR
- 背景
- 关键发现
- 证据
- 风险/限制
- 延伸阅读
- 可引用段落锚点



这类页面天然适合被其他 LLM 读取和引用。

2.4 交互式解释模式

适合复杂概念、教程、可视化笔记。

输出特征：

- 折叠面板
- 图表
- 步骤演示
- Mermaid 图
- 数学公式
- 交互式 checklist
- 局部代码运行区，可选



这个模式可以把普通 Markdown 变成更像 “mini interactive article” 的页面。

2.5 知识库模式

适合一组笔记，而不是单篇文章。

输出特征：

- 左侧导航
- 标签系统
- 双链
- 相关笔记
- 全文搜索
- 反向链接
- 知识图谱，可选



这个可以对标 Obsidian Publish、Quartz、Docusaurus，但更强调 LLM-readable。

2.6 展示页模式

适合项目介绍、作品集、报告首页。

输出特征：

- Hero section
- 卡片式模块
- 时间线
- 数据展示
- CTA
- 图文混排



这可以让用户把一份 DOCX/Markdown 项目说明变成一个可分享 landing page。

3. 转换流程应该分两层：规则转换 + LLM 增强

不要让 LLM 直接从原始文件生成最终 HTML。那样不可控、成本高，也容易产生幻觉。

更稳的流程是：

1. 解析原始文件
2. 转换成中间格式
3. 提取结构
4. 调用 LLM 做增强
5. 套用 HTML 预设模板
6. 安全检查
7. 生成最终 artifact



具体可以这样：

```
md/docx/html/txt
↓
Normalized Markdown / AST
↓
Document Structure
- title
- headings
- paragraphs
- tables
- code blocks
- images
- references
↓
LLM Enhancement
- 摘要
- 标签
- 目录优化
- 章节重组
- 重点提取
- FAQ
- 术语表
```



↓
HTML Renderer
↓
Sandbox + Validation
↓
Published Artifact

中间格式非常重要。建议项目内部不要直接操作 HTML 字符串，而是维护一个统一的 Note AST 或 Artifact AST。

例如：

```
</> JSON   
  
{  
  "title": "Transformer 学习笔记",  
  "blocks": [  
    {  
      "type": "heading",  
      "level": 1,  
      "text": "Attention Mechanism"  
    },  
    {  
      "type": "paragraph",  
      "text": "Attention allows the model to..."  
    },  
    {  
      "type": "code",  
      "language": "python",  
      "content": "..."  
    }  
  ],  
  "metadata": {  
    "tags": ["AI", "Transformer", "Deep Learning"],  
    "source_format": "docx"  
  }  
}
```

这样后续可以稳定地转换成：

HTML 
Markdown
Text
JSON
Chunks
RSS
llms.txt index
MCP-readable resource

4. 自定义转换预设可以成为项目生态

用户自定义转换是很重要的功能，但要设计成“安全、可复用、可分享”的 preset，而不是只让用户随便写 prompt。

一个预设可以包含：

</> YAML



```
name: research-brief
label: Research Brief
description: Convert notes into a structured research memo.

input:
  formats: ["md", "docx", "html", "txt"]

output:
  layout: "article"
  theme: "serif-clean"
  components:
    - toc
    - summary_card
    - key_findings
    - quote_blocks
    - references
    - related_notes

llm:
  mode: "faithful"
  tasks:
    - summarize
    - extract_key_points
    - generate_tags
    - generate_faq
  prompt: |
    Convert the source note into a structured research brief.
    Preserve all factual claims from the original source.
    Do not invent citations, numbers, names, or conclusions.

safety:
  allow_scripts: false
  allow_iframes: false

export:
  markdown: true
  json: true
  llm_chunks: true
```

这意味着预设本身可以开源、版本化、分享。

未来可以形成：

preset marketplace / preset gallery



用户可以安装别人做好的转换风格：

`</> Bash`



```
artifactpress preset install academic-paper
artifactpress preset install visual-learning-note
artifactpress preset install startup-memo
```

5. 转换模式需要区分“忠实转换”和“创意增强”

这是产品上必须明确的地方。

因为 LLM 转换文档时容易出现一个问题：它可能会改写、添加、删减内容。

所以建议提供三种转换模式。

5.1 Faithful Mode

忠实模式。

适合：

论文笔记
法律/合同摘录
学术内容
严肃研究
技术文档



规则：

- 不新增事实
- 不编造引用
- 不改变原文结论
- 只优化排版和结构
- 所有新增摘要必须标记为 AI-generated



5.2 Enhanced Mode

增强模式。

适合：

学习笔记
课程总结



个人知识库
技术教程

规则：

- 可以重组结构
- 可以生成摘要
- 可以生成 FAQ
- 可以补充解释性过渡语
- 不能编造具体事实



5.3 Creative Mode

创意模式。

适合：

作品展示
营销页
个人主页
项目 landing page



规则：

- 可以大幅重写表达
- 可以调整语气
- 可以生成视觉化布局
- 需要用户确认后发布



这三个模式会让用户更放心，也能避免“AI 把我的原始笔记改坏了”。

6. 支持的文件类型可以分阶段做

MVP 阶段

先支持：

.md
.html
.txt
.docx



这四个就够了。

原因：

- Markdown 是 AI 输出和个人笔记的主流格式；
- HTML 是项目最初的核心；
- TXT 简单但常见；
- DOCX 覆盖普通用户、学生、研究者和办公场景。

第二阶段

再支持：

```
.pdf
.notion export
.obsidian vault
.ipynb
.csv
```



PDF 不建议第一阶段做得太重。PDF 解析质量不稳定，图表、公式、双栏排版、扫描件都会带来复杂性。可以先只支持 text-based PDF，扫描件 OCR 放到后面。

7. 用户界面可以这样设计

用户操作路径：

1. 上传文件
2. 系统解析内容
3. 选择转换预设
4. 选择转换模式
5. 预览
6. 对局部内容进行调整
7. 发布



页面上可以有几个核心控件：

```
Input File
- source.md
- source.docx
- source.html
```

```
Preset
- Clean Article
- Study Note
- Research Brief
- Interactive Explainer
- Knowledge Base Page
- Landing Page
```

```
Conversion Mode
- Faithful
```



- Enhanced
- Creative

Output

- Public HTML
- Markdown
- Text
- JSON
- LLM chunks

预览界面最好支持三栏：

- 左侧：原始内容
- 中间：结构化中间稿
- 右侧：最终 HTML 预览



这样用户可以清楚看到：

- LLM 改了什么；
- 哪些内容被提炼了；
- 哪些内容被加入了摘要；
- 最终页面长什么样。

8. 项目核心架构可以更新为

artifactpress/	
apps/	
web/	Web 管理后台
demo-site/	示例站点
packages/	
core/	核心数据模型
ingest/	文件导入
parser-md/	Markdown 解析
parser-docx/	DOCX 解析
parser-html/	HTML 解析与清洗
transformer/	AST 转换
llm/	LLM 调用、prompt、结构化输出
presets/	官方转换预设
renderer/	HTML 渲染器
sandbox/	HTML 安全沙箱
search/	搜索索引
mcp-server/	MCP 接口
cli/	命令行工具
theme-default/	默认主题
templates/	
clean-article/	

```
study-note/  
research-brief/  
interactive-explainer/  
knowledge-base/  
  
examples/  
  markdown-note/  
  docx-report/  
  html-artifact/
```

CLI 可以这样：

❏ Bash

```
artifactpress convert ./notes/transformer.md --preset study-note  
artifactpress convert ./report.docx --preset research-brief --mode faithful  
artifactpress publish ./output/transformer
```

或者一步完成：

❏ Bash

```
artifactpress publish ./report.docx \  
  --preset research-brief \  
  --mode faithful \  
  --visibility public
```

9. LLM 接口不要只用于读取，也要用于转换

之前的项目重点是“让 LLM 读取网站内容”。

现在可以加一层：

让 LLM 参与内容生产，但输出必须可追踪、可回滚、可复现。

LLM 在项目中可以承担这些任务：

- 文档摘要
- 标题优化
- 标签生成
- 目录生成
- 章节重组
- 术语表提取
- FAQ 生成
- 相关笔记推荐
- Markdown → HTML layout planning

- DOCX → structured article
- source note → learning card

但是 LLM 不应该直接拥有最终控制权。

建议设计成：

LLM proposes.
Renderer builds.
User approves.
System publishes.



也就是 LLM 负责“建议结构”，真正输出 HTML 的是项目自己的 renderer。

这样生成结果更稳定，也更安全。

10. 这个功能会形成很好的开源生态

一旦支持自定义转换预设，社区可以贡献：

- 学术论文笔记预设
- 数学学习笔记预设
- 编程教程预设
- 读书笔记预设
- 会议纪要预设
- 产品需求文档预设
- 投资研究笔记预设
- 个人作品集预设
- 课程页面预设
- 技术文档预设



项目本体提供基础设施，社区提供风格和工作流。

这比单纯做一个博客系统更有扩展性。

11. 修正后的项目一句话

可以重新定义为：

ArtifactPress 是一个开源的 AI-native 笔记发布系统，可以把 Markdown、DOCX、HTML 等文件通过转换预设发布成高质量 HTML 页面，并自动生成适合 LLM 读取的结构化内容接口。

更短一点：

把任意笔记变成可发布、可分享、可被 AI 读取的网页。

英文可以是：

An open-source AI-native publishing engine for turning notes into beautiful, LI 

12. MVP 可以改成这个版本

第一版建议聚焦：

输入：

- Markdown
- HTML
- DOCX
- TXT

转换：

- Clean Article
- Study Note
- Research Brief

模式：

- Faithful
- Enhanced

输出：

- HTML 页面
- Markdown
- Text
- JSON metadata
- llms.txt
- sitemap



暂时不要做：

- PDF OCR
- 多人协作
- 在线富文本编辑器
- 复杂插件市场
- 社交网络
- 自动训练个人模型



第一版只要把“上传一个 DOCX/Markdown → 选择预设 → 生成漂亮 HTML → 发布 → LLM 可读”跑通，就已经很有辨识度。

13. 这个补充让项目更有壁垒

单纯托管 HTML，容易被理解成：

GitHub Pages + 上传按钮



但加入多格式输入和 LLM 转换预设后，项目变成：

AI 时代的个人知识发布编译器



它的壁垒来自：

- 多格式解析
- 文档结构化
- LLM 转换预设
- 安全 HTML 渲染
- 可复现的发布流水线
- LLM-readable 输出
- 开源 preset 生态



这比“新型博客”更准确，也更有长期价值。